CKKS-双服务器/层次型结构扩展

🥇 其实，你只需要建立一个最核心的物理直觉：**模数就是“数字容器的最大容量（天花板）”，而缩放因子 Δ 就是“把有效数据抬高多少倍，以便在底部腾出空间装噪声”**。

分圆多项式环 $\mathcal{R}_M = \mathbb{Z}[X]/(X^M+1)$（其中 M 是 2 的幂次，如 4096 或 8192()）上的 **RLWE（环上带误差学习）**

底层基石：RLWE 里的“模数 Q”到底是什么？

在分圆多项式环 $\mathcal{R}_{M,Q} = \mathbb{Z}_Q[X]/(X^M+1)$ 中，包含两个核心维度参数：

1. **多项式次数 M（常记为 n 或 N）**：决定了多项式有多少个系数（即能打包多少个数据槽），比如 $M = 4096$。
2. **系数模数 Q（常记为 q）**：决定了多项式的**每一个整数系数能有多大**。所有系数的加减乘运算，都要对 Q 取模（即落在 $[-Q/2, Q/2)$ 之间）。如果 $\log_2 Q = 218$ bits()，就意味着每个系数都是一个最高 218 位的超大整数。

RLWE 困难性问题：给定一个在 $\mathcal{R}_{M,Q}$ 上均匀随机采样的多项式 a，一个系数极小的私钥多项式 $s \in \{-1,0,1\}^M$ ()，以及一个系数极小的高斯噪声多项式 e（标准差通常为 $\sigma \approx 3.2$ ()），构造出的二元组：

$$(b = -a \cdot s + e \pmod{Q},\quad a) \in \mathcal{R}_{M,Q}^2$$

在计算上与完全均匀随机的两个多项式不可区分。

- **安全矛盾点（模数 Q 不能乱选的根源）**：如果固定环维度 M，**模数 Q 越大，噪声占比 e/Q 就越小，密码系统反而越不安全（越容易被格基规约算法攻破）！但模数 Q 越大，容器天花板越高，能容纳的同态运算次数（噪声增长空间）就越多。**

二、 BFV 算法与它的双模数体系（P 与 Q）

BFV 是“精确整数同态加密”()。它的核心思想是：**高位存明文，低位存噪声（MSB 编码）**，解密时通过整数除法加上四舍五入，把低位噪声**100% 彻底抹除**()。

1.1 BFV 的三个核心参数：P, Q, Δ

- **明文模数 P（常记为 t）**：你真正想计算的业务整数的范围。明文多项式 $m \in \mathcal{R}_{M,P}$，其系数都在 $[0, P)$ 内（例如设 $P = 65537$）。
- **密文模数 Q（常记为 q）**：加密后密文多项式所在的巨大容器容量，满足 $Q \gg P$。
- **缩放因子 $\Delta = \lfloor Q/P \rfloor$**：把明文 m 乘以 Δ，相当于在二进制下把明文 m **向左平移到容器 Q 的最高位（顶部）**，从而在底部腾出大小为 Δ 的空白隔间来容纳噪声！

1.2 BFV 的加解密公式流（看清模数如何工作）

- **公钥生成**：采样私钥 $s \leftarrow \chi_s$，公共随机多项式 $a \leftarrow \mathcal{R}_{M,Q}$，小噪声 $e \leftarrow \chi_e$，输出公钥：

$$pk = (p_0, p_1) = (-a \cdot s + e \pmod{Q}, \quad a)$$

- **加密** $\mathrm{BFV.Enc}(pk, m)$：输入明文 $m \in \mathcal{R}_{M,P}$，采样随机小多项式 $u \leftarrow \chi_s$ 和小噪声 $e_0, e_1 \leftarrow \chi_e$，输出密文 $CT = (c_0, c_1) \in \mathcal{R}^2_{M,Q}$：

$$\begin{cases} c_0 = p_0 \cdot u + \mathbf{\Delta} \cdot \mathbf{m} + e_0 \pmod{Q} \\ c_1 = p_1 \cdot u + e_1 \pmod{Q} \end{cases}$$

- **解密** $\mathrm{BFV.Dec}(s, CT)$：**第一步（消去掩码，提取带噪明文** $b \in \mathcal{R}_{M,Q}$**）**：

$$b = c_0 + c_1 \cdot s \pmod{Q} = (-asu + eu + \Delta \cdot m + e_0) + (au + e_1)s = \mathbf{\Delta} \cdot \mathbf{m} + \underbrace{(\mathbf{eu} + \mathbf{e_0} + \mathbf{e_1 s})}_{\mathbf{e_{CT}}} \pmod{Q}$$

- **第二步（缩放取整，抹除噪声还原回** $\mathcal{R}_{M,P}$**）**：

$$\hat{m} = \left\lfloor \frac{P}{Q} \cdot b \right\rceil \pmod{P} \approx \left\lfloor \frac{\Delta \cdot m + e_{CT}}{\Delta} \right\rceil = m + \left\lfloor \frac{e_{CT}}{\Delta} \right\rceil$$

> **正确性条件**：只要累积噪声的无穷范数满足 $\|e_{CT}\| < \Delta/2$，那么 $\lfloor e_{CT}/\Delta \rceil$ 就严格等于 **0**！噪声被完全消除，可精确恢复出明文 m。

(这就解释了为什么第一篇论文 ATASSES 必须用 BFV 来保护解密份额 s_i：因为外层解密份额 s_i 本身是模 Q 下的精确多项式，ATASSES 把外层的 Q 当作内层 BFV 的明文模数 P'，再选一个更大的内层密文模数 $Q' > 2N(P')^2 B'$，就能利用内层 $\lfloor \cdot/\Delta' \rceil$ 把被拉格朗日系数放大的内层加密底噪 $L_i e'_{i,1}$ 彻底归零！)

在联邦学习中，梯度 $\mathbf{g} = (0.0123, -0.0456, \dots)$ 是实数/浮点数。如果用 BFV，每次乘法后定点数的放大倍数都会平方膨胀（如 deg^2），极易溢出明文模数 P。为此，**CKKS 提出了“近似同态加密”：砍掉独立的明文模数 P，把噪声视为浮点数自带的微小舍入误差，让明文和噪声住在同一个屋檐下！**

1.3 CKKS 的核心参数：没有明文模数 P，只有缩放因子 Δ 与模数链 Q_l

- **编码缩放因子 Δ（常记为 p 或 Δ_{ckks}）**：比如设 $\Delta = 2^{40}$ 或 2^{50}。它的作用不是把明文推到 Q 的顶部，而是**把小数点后 40 位（约 12 位十进制有效数字）的浮点数放大成整数**，即 $m(X) = \lfloor \Delta \cdot \phi^{-1}(\mathbf{g}) \rceil$。
- **密文模数链 $Q_L > Q_{L-1} > \cdots > Q_0$**： 为什么 CKKS 的模数 Q 要设计成一串素数的乘积 $Q_L = q_0 \cdot q_1 \cdots q_L$？
  - 假设两个放大了 Δ 倍的明文相乘：$(\Delta \cdot m_1) \times (\Delta \cdot m_2) = \Delta^2 \cdot (m_1 m_2)$。如果不处理，几次乘法后数值就会超过模数上限（超过 Q_L）。
  - 因此 CKKS 引入了 **重缩放（Rescaling）** 操作：每次做完乘法，直接把密文除以当前层的素数 $q_l \approx \Delta$，同时**把密文模数也同步降低一层**（从 Q_l 降到 $Q_{l-1} = Q_l / q_l$：
  - $\overline{ct}' = \left\lfloor q_l^{-1} \cdot \overline{ct} \right\rceil \pmod{Q_{l-1}}$
  - 这样，内部编码明文就从 $\Delta^2 m_1 m_2$ 重新变回了 $\Delta m_1 m_2$，保持缩放因子 Δ 恒定不变！

1.4 CKKS 的加解密公式流

- **编码与加密** $\mathsf{CKKS.Enc}(pk, \mathbf{g})$： 先将浮点向量 $\mathbf{g} \in \mathbb{C}^{M/2}$ 编码为整数系数多项式 $m(X) = \lfloor \Delta \cdot \phi^{-1}(\mathbf{g}) \rceil \in \mathcal{R}_{M,Q}$，然后直接与掩码和噪声相加（**注意：$m(X)$ 前面不再乘 $\lfloor Q/P \rfloor$，而是直接裸加**）：
- $$\begin{cases} c_0 = p_0 \cdot u + \mathbf{m}(\mathbf{X}) + e_0 \pmod{Q} \\ c_1 = p_1 \cdot u + e_1 \pmod{Q} \end{cases}$$
- **解密与解码** $\mathsf{CKKS.Dec}(s, CT)$：

第一步（消去掩码）：

- $b = c_0 + c_1 \cdot s \pmod{Q} = m(X) + e_{CT} = \lfloor \Delta \cdot \phi^{-1}(\mathbf{g}) \rceil + e_{CT} \pmod{Q}$
- *(条件：只要 $|m(X) + e_{CT}| < Q/2$，就不会发生模 Q 环绕溢出)。*

第二步（解码还原浮点数）：

- $\hat{\mathbf{g}} = \Delta^{-1} \cdot \phi(b) = \mathbf{g} + \dfrac{\phi(e_{CT})}{\Delta}$
- 因为 e_{CT} 只有几十（约 2^5），而 $\Delta = 2^{40} \sim 2^{50}$，所以 $\dfrac{\phi(e_{CT})}{\Delta} \approx 10^{-9}$！虽然噪声没像 BFV 那样被归零，但它被缩小到小数点后第 9 位，对联邦学习梯度的精度没有影响()。

在将 **外层 CKKS 梯度加密** 与 **内层 ATASSES（由两把子密钥 S_1, S_2 合成联合解密密钥的机制）** 深度融合的联邦学习场景下，针对 ① **分簇层次性架构（Hierarchical Architecture）** 和 ② **非共谋双服务器架构（Twin-Server Architecture）** 的完整威胁模型、方案流程、实体交互流程及闭环数学公式推导如下。

- **全局统一符号约定：**

  - **外层 CKKS 空间（加密浮点梯度）：**
  - 分圆多项式环 $\mathcal{R}_{M,Q} = \mathbb{Z}_Q[X]/(X^M+1)$
  - CKKS 浮点编码缩放因子为 Δ_{ckks}。
  - 全局外层总私钥为 $S \in \mathcal{R}_{M,Q}$
  - 外层全局公钥为 $PK = (p_0, p_1) = (-a \cdot S + e_{pk},\ a) \in \mathcal{R}^2_{M,Q}$。

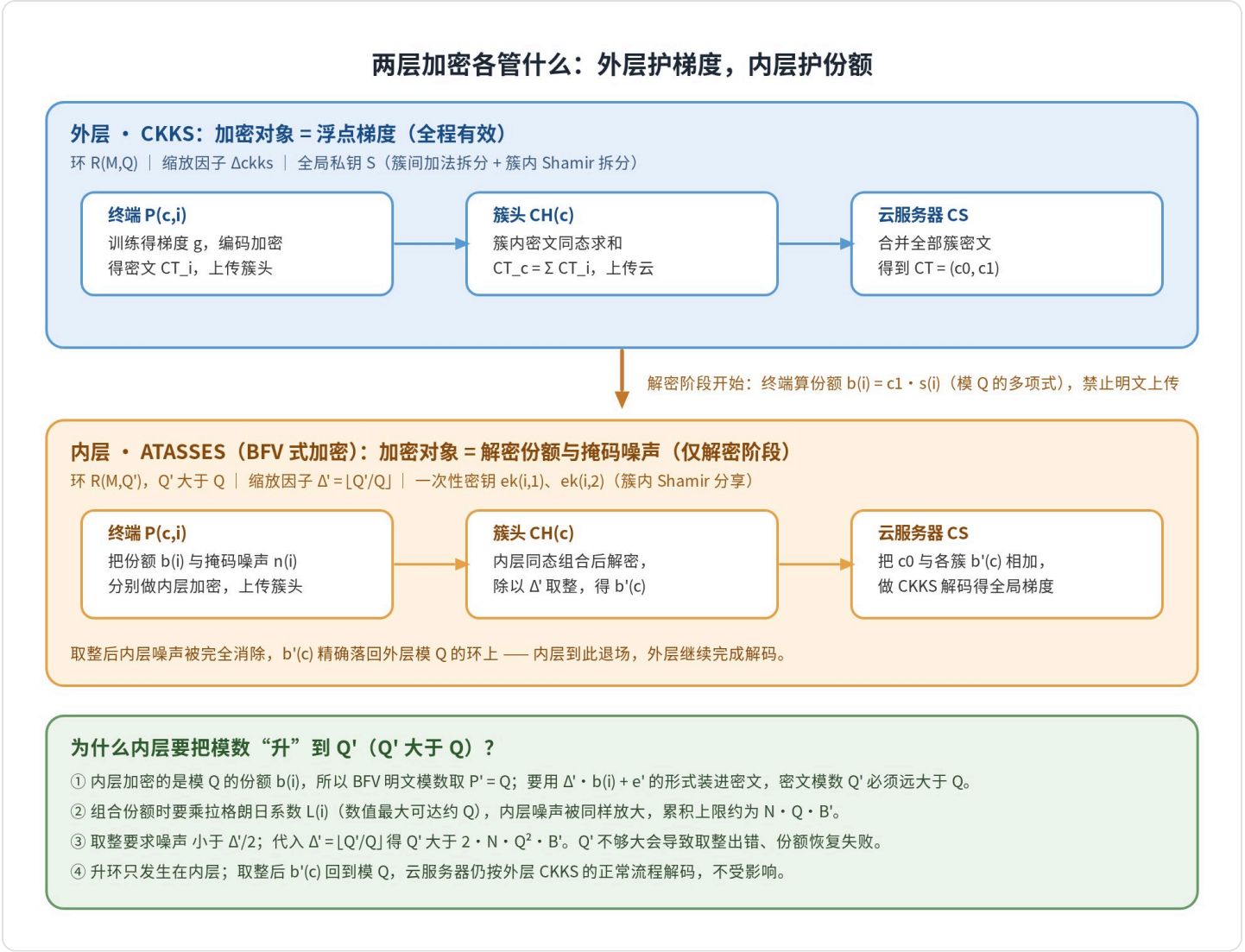
  - **内层 ATASSES 空间（保护部分解密份额）**：僅在方案一的簇内使用模数更大的环 $\mathcal{R}_{M,Q'}$，其中内层明文模数为 $P' = Q$，内层密文模数为 Q'，内层整数缩放因子为 $\Delta' = \lfloor Q'/Q \rfloor$；方案二直接复用外层环 $\mathcal{R}_{M,Q}$（无模数膨胀）。

三、 外层与内层：怎么区分，以及为什么要“升环”

这两个词之所以容易混，是因为它们**加密的对象完全不同**，不在同一个阶段起作用：

- **外层（CKKS）**：保护的是**梯度数据**。每个终端把本地浮点梯度用 CKKS 加密后上传，簇头、云服务器只做同态加法，从训练上传一直用到最终解码。这一层解决“别人看不到我的梯度”。
- **内层（ATASSES，BFV 式加密）**：保护的是**部分解密份额**。解密阶段每个终端要计算 $b_{c,i} = c_1 \cdot s_{c,i}$ 交给簇头，如果直接明文上传，簇头可以拿多个份额反推出私钥 $s_{c,i}$。所以要给份额再套一层 BFV 式加密，让簇头只能拿到解密结果、拿不到份额本身。这一层只在解密的两个回合里存在，解决“别人看不到我的密钥份额”。

一句话记忆：**外层管“数据”（梯度），内层管“钥匙”（解密份额）**。内层加密结束后份额会取整还原成模 Q 的多项式，再交给云服务器，此时内层就退场了。



示意图与公式推导的对应关系：

- 上方蓝色带 = **外层**：对应“阶段 1：本地训练、外层 CKKS 梯度加密与两级密态聚合”，产出全局密文 $CT = (c_0, c_1)$。
- 下方橙色带 = **内层**：对应“阶段 2 / 阶段 3”这两个回合，终端把份额 $b_{c,i}$ 与掩码噪声 $n_{c,i}$ 分别加密，簇头组合后解密取整得到 $b'_c = c_1 \cdot S^{(c)} + \sum_i n_{c,i}$。
- 最后回到外层：对应“阶段 4”，云服务器计算 $c_0 + \sum_c b'_c$ 并做 CKKS 解码。

外层与内层对照表

对比项	外层（CKKS）	内层（ATASSES，BFV 式）
加密对象	浮点梯度 $\mathbf{g}$	部分解密份额 $b_{c,i}$ 与掩码噪声 $n_{c,i}$
使用阶段	全程：训练上传 → 密文聚合 → 最终解码	仅解密阶段（阶段 2–3 两个回合）
多项式环	$\mathcal{R}_{M,Q}$	方案一用 $\mathcal{R}_{M,Q'}$（$Q' > Q$）；方案二复用 $\mathcal{R}_{M,Q}$

缩放与取整	Δ_{ckks} 放大浮点数，解码后保留约 10^{-9} 量级的微小误差	$\Delta' = \lfloor Q'/Q \rceil$ 放大整数，解密取整后噪声 100% 消除
密钥	全局总私钥 S ，簇间加法拆分 + 簇内 Shamir 拆分	每轮一次性密钥 $ek_{c,i,1}, ek_{c,i,2}$ ，仅簇内 Shamir 分享
保护目标	梯度数据的机密性	私钥份额的机密性（防止从份额反推密钥）

为什么要强调“升环”这一点？

因为它是方案一**正确性的硬性参数条件**，也是方案一与方案二**开销差异的根源**，在论文（ATASSES，USENIX Security 2025）里对应“选取略大的密文空间”这一关键设计：

1. **正确性**：内层解密依赖“除以 Δ' 后四舍五入”来彻底消除噪声。拉格朗日系数会把内层噪声放大约 $N \cdot Q$ 倍，若 Q' 不满足 $Q' > 2NQ^2B'$ ，取整这一步就会出错，恢复出的份额是错的，整个解密协议失败。
2. **开销**：Q' 比 Q 大，意味着内层密文比外层密文更大、计算更贵。这是方案一为“防份额泄露”付出的代价，文档里必须写清楚，否则读者无法评估通信与计算成本。
3. **与方案二对照**：方案二（双服务器）不需要升环——掩码密钥由短种子经 PRG 展开，掩码本身是均匀随机的、不携带 BFV 噪声，全程直接在外层环 $\mathcal{R}_{M,Q}$ 上运算。指出这一点，才能说明为什么方案二的通信开销更小。

方案一：面向 CKKS 的分簇层次性架构（Edge-Assisted Hierarchical CKKS-ATASSES）

2. 威胁模型（Threat Model）

1. **系统拓扑与实体**：系统包含一个云服务器（Cloud Server, CS ）、C 个边缘服务器/簇头（Edge Servers / Cluster Heads, $\mathrm{CH}_c, c \in [1, C]$ ），以及分属各簇的 N 个终端设备（Clients $P_{c,i}, i \in [1, n_c]$ ，满足 $\sum_{c=1}^{C} n_c = N$ ）。

2. **异步掉线容忍（Dropout Resilience）**：在每一轮解密阶段，允许第 c 簇内最多有 $n_c - t_c$ 个终端因移动或网络故障随时掉线（即每簇至少有 t_c 个终端在线）；同时允许簇内第一回合上传密文的在线集合 $\mathcal{T}_{c,1}$ 与第二回合上传密钥份额的在线集合 $\mathcal{T}_{c,2}$ 不一致（僅需满足 $|\mathcal{T}_{c,1}| \geq t_c, |\mathcal{T}_{c,2}| \geq t_c$ ）。

3. **腐化与合谋能力（Semi-honest & Collusion）**：CS、所有簇头 CH_c 以及被腐化的终端均为半诚实（Honest-but-Curious）敌手，严格遵守协议执行计算，但试图从交互视图（Views）中利用梯度反演攻击（如 DLG）推断诚实终端的私有梯度 $\mathbf{g}_{c,i}$ 或恢复外层全局总私钥 S。允许 CS、任意数量的簇头 CH_c 以及每个簇内最多 $t_c - 1$ 个被腐化终端联合合谋（即每簇至少存在 1 个诚实在线终端）。

2. 方案流程与闭环公式推导

阶段 0：系统初始化与密钥分享（`Hier-CKKS-Setup`）

1. **外层全局总私钥 S 的簇间加法拆分与簇内 Shamir 拆分**：

全局外层总私钥 $S \in \mathcal{R}_{M,Q}$ 在逻辑上按**簇间加法秘密共享**拆分为 C 个簇级子私钥 $S^{(c)}$：

$$S = \sum_{c=1}^{C} S^{(c)} \pmod{Q}$$

在第 c 个簇内部，簇级子私钥 $S^{(c)}$ 通过环 $\mathcal{R}_{M,Q}$ 上的 (t_c, n_c)-Shamir 秘密共享（基于满足 Exceptional Sequence 的公开评估点 $\{\alpha_{c,i}\}_{i=1}^{n_c}$）分发给簇内终端 $P_{c,i}$。即存在 $t_c - 1$ 次多项式 $F_c(X)$（常数项 $F_c(0) = S^{(c)}$），使得终端 $P_{c,i}$ 持有外层私钥份额 $s_{c,i} = F_c(\alpha_{c,i}) \in \mathcal{R}_{M,Q}$。

2. **两级私钥重构闭环恒等式**：对于第 c 簇内任意大小 $|\mathcal{T}_{c,1}| \geq t_c$ 的在线集合，其局部拉格朗日插值系数 $L_{c,i}^{(\mathcal{T}_{c,1})} = \prod_{j \in \mathcal{T}_{c,1}, j \neq i} \frac{\alpha_{c,j}}{\alpha_{c,j} - \alpha_{c,i}} \pmod{Q}$ 严格满足：

$$\sum_{i \in \mathcal{T}_{c,1}} L_{c,i}^{(\mathcal{T}_{c,1})} \cdot s_{c,i} = S^{(c)} \pmod{Q} \implies \sum_{c=1}^{C} \sum_{i \in \mathcal{T}_{c,1}} L_{c,i}^{(\mathcal{T}_{c,1})} \cdot s_{c,i} = \sum_{c=1}^{C} S^{(c)} = S \pmod{Q}$$

阶段 1：本地训练、外层 CKKS 梯度加密与两级密态聚合（`Outer-CKKS-Agg`）

1. **终端本地训练与 CKKS 编码加密**：终端 $P_{c,i}$ 训练得到本地浮点梯度向量 $\mathbf{g}_{c,i} \in \mathbb{R}^{M/2}$（若梯度总长 $K \gg M/2$，则切分为 $\tau = \lceil 2K/M \rceil$ 个子向量并行打包，此处以单个打包块为例）。$P_{c,i}$ 使用 CKKS 逆正则嵌入 ϕ^{-1} 和缩放因子 Δ_{ckks} 将其**编码**为多项式：

$$m_{c,i}(X) = \lfloor \Delta_{ckks} \cdot \phi^{-1}(\mathbf{g}_{c,i}) \rceil \in \mathcal{R}_{M,Q}$$

随后采样随机多项式 $u_{c,i} \longleftarrow \chi_s$ 和噪声 $e_{c,i,0}, e_{c,i,1} \longleftarrow \chi_e$，用全局公钥 $PK = (p_0, p_1)$ 执行标准 CKKS 公钥**加密**：

$$CT_{c,i} = \begin{cases} c_{c,i,0} = p_0 \cdot u_{c,i} + m_{c,i}(X) + e_{c,i,0} \pmod{Q} \\ c_{c,i,1} = p_1 \cdot u_{c,i} + e_{c,i,1} \pmod{Q} \end{cases}$$

1. **边缘簇头与云服务器两级密文同态累加：**

   - $P_{c,i}$ 将 $CT_{c,i}$ 上传给所属簇头 CH_c，CH_c 聚合本簇密文得到 $CT_c = \sum_i CT_{c,i}$ 并上传给云服务器 CS。

   - CS 汇总所有簇的密文，得到外层全局梯度密文 $CT = (c_0, c_1) = \sum_{c=1}^{C} CT_c$：

   - $$\begin{cases} c_0 = p_0 \sum_{c,i} u_{c,i} + \sum_{c,i} m_{c,i}(X) + \sum_{c,i} e_{c,i,0} \pmod{Q} \\ c_1 = p_1 \sum_{c,i} u_{c,i} + \sum_{c,i} e_{c,i,1} \pmod{Q} \end{cases}$$

   - 记全局编码明文和为 $M_{agg}(X) = \sum_{c,i} m_{c,i}(X)$。将 $p_0 = -a \cdot S + e_{pk}$ 和 $p_1 = a$ 代入，外层 CKKS 解密核心关系为：

   - $$c_0 + c_1 \cdot S = M_{agg}(X) + \underbrace{\left(e_{pk} \sum_{c,i} u_{c,i} + \sum_{c,i} e_{c,i,0} + S \sum_{c,i} e_{c,i,1} \right)}_{\text{外层 CKKS 密文累积噪声 } e_{CT}} \pmod{Q}$$

   - CS 将公共密文组件 $c_1 \in \mathcal{R}_{M,Q}$ 下发给各簇头 CH_c 并广播至簇内终端，请求协同计算 $b = c_1 \cdot S \pmod{Q}$。

阶段 2：簇内 ATASSES 第一回合 —— 内层加密保护份额与噪声（`Cluster-R1`）

第 c 簇的在线终端 $P_{c,i}$ 收到 c_1 后，在簇内启动基于两把子密钥 $(S_{in,1}^{(c)}, S_{in,2}^{(c)})$ 的 ATASSES 第一回合：

1. **计算外层局部解密份额：**

$$b_{c,i} = c_1 \cdot s_{c,i} \pmod{Q}$$

由簇内线性性质，必有 $\sum_{i\in\mathcal{T}_{c,1}} L_{c,i}^{(\mathcal{T}_{c,1})}\cdot b_{c,i} = c_1\cdot S^{(c)} = b_c \pmod{Q}$（其中 b_c 为第 c 簇的目标解密分量）。

2. **生成簇内一次性加密密钥并在簇内做 Shamir 分享**：在内层更大的模数 $\mathcal{R}_{M,Q'}$ 上（其中 $\Delta' = \lfloor Q'/Q \rfloor$），$P_{c,i}$ 采样两把内层子私钥 $ek_{c,i,1}, ek_{c,i,2} \longleftarrow \chi'_{sk}$，仅在**本簇内部的 n_c 个节点间**做 (t_c, n_c)-Shamir 共享，向本簇节点 $P_{c,j}$ 发送切片 $(ekShare_{j,i,1}^{(c)},\ ekShare_{j,i,2}^{(c)})$。

3. **把解密份额和掩码噪声分别做内层加密（内层密文使用模数 Q' 的环）**：$P_{c,i}$ 采样用于掩盖外层噪声的掩码噪声 $n_{c,i} \longleftarrow [-B_{sm}, B_{sm}]$，采样内层小加密底噪 $e'_{c,i,1}, e'_{c,i,2} \longleftarrow \psi'$（范数受限于 B'），利用簇内 CRS 生成的公共基底 $p'_1 u'_c = -a'_c u'_c \in \mathcal{R}_{M,Q'}$ 分别加密 $b_{c,i}$ 和 $n_{c,i}$ ()：

$$CTb_{c,i} = \begin{cases} c'_{b,c,i,0} = (a'_c \cdot ek_{c,i,1})u'_c + \Delta'\cdot b_{c,i} + e'_{c,i,1} \\ c'_{c,1} = -a'_c \cdot u'_c \end{cases} \pmod{Q'}$$

$$CTn_{c,i} = \begin{cases} c'_{n,c,i,0} = (a'_c \cdot ek_{c,i,2})u'_c + \Delta'\cdot n_{c,i} + e'_{c,i,2} \\ c'_{c,1} = -a'_c \cdot u'_c \end{cases} \pmod{Q'}$$

$P_{c,i}$ 将 $(CTb_{c,i}, CTn_{c,i})$ 上传给本簇簇头 CH_c。CH_c 收齐本簇第一回合在线集合 $\mathcal{T}_{c,1}$（$|\mathcal{T}_{c,1}| \geq t_c$）的密文后，计算并向簇内广播拉格朗日系数 $\{L_{c,i}^{(\mathcal{T}_{c,1})}\}_{i\in\mathcal{T}_{c,1}}$。

阶段 3：簇内 ATASSES 第二回合 —— 簇头解密取整得到簇级份额 b'_c（`Cluster-R2`）

1. **终端本地计算联合解密密钥的份额**：本簇第二回合在线终端 $P_{c,j} \in \mathcal{T}_{c,2}$（$|\mathcal{T}_{c,2}| \geq t_c$）收到 $\{L_{c,i}^{(\mathcal{T}_{c,1})}\}$ 后，在本地分别计算本簇**第一把聚合子密钥** $S_{in,1}^{(c)} = \sum_{i\in\mathcal{T}_{c,1}} L_{c,i}^{(\mathcal{T}_{c,1})} ek_{c,i,1}$ 的份额与**第二把聚合子密钥** $S_{in,2}^{(c)} = \sum_{i\in\mathcal{T}_{c,1}} ek_{c,i,2}$ 的份额，并将二者相加绑定后发给簇头 CH_c ()：

1. $$dkShare_{c,j} = \underbrace{\sum_{i\in\mathcal{T}_{c,1}} L_{c,i}^{(\mathcal{T}_{c,1})}\cdot ekShare_{i,j,1}^{(c)}}_{S_{in,1}^{(c)}\text{ 的份额}} + \underbrace{\sum_{i\in\mathcal{T}_{c,1}} ekShare_{i,j,2}^{(c)}}_{S_{in,2}^{(c)}\text{ 的份额}} \pmod{Q'}$$

2. **簇头 CH_c 插值恢复联合总私钥并执行内层解密取整**：

- CH_c 利用集合 $\mathcal{T}_{c,2}$ 发来的 $dkShare_{c,j}$ 执行 Shamir 插值，恢复出本簇联合内层解密私钥：

$$dk_c = S_{in,1}^{(c)} + S_{in,2}^{(c)} = \sum_{i \in \mathcal{T}_{c,1}} \left(L_{c,i}^{(\mathcal{T}_{c,1})} \cdot ek_{c,i,1} + ek_{c,i,2} \right) \pmod{Q'}$$

- CH_c 将通道 1 密文乘系数 $L_{c,i}^{(\mathcal{T}_{c,1})}$，与通道 2 密文直接相加，合成簇内总密文首项：

$$C'_{all,c,0} = \sum_{i \in \mathcal{T}_{c,1}} \left(L_{c,i}^{(\mathcal{T}_{c,1})} \cdot c'_{b,c,i,0} + c'_{n,c,i,0} \right) = (a'_c u'_c) \cdot (S_{in,1}^{(c)} + S_{in,2}^{(c)}) + \Delta' \cdot \left(c_1 \cdot S^{(c)} + \sum_{i \in \mathcal{T}_{c,1}} n_{c,i} \right) + E'_{total,c} \pmod{Q'}$$

- CH_c 用 $dk_c = S_{in,1}^{(c)} + S_{in,2}^{(c)}$ 消去掩码项 $(a'_c u'_c) dk_c$，并除以 Δ' 四舍五入取整。由于簇内节点数仅为 $n_c = N/C$，只需满足内层模数 $Q' > 2 n_c Q^2 B'$（使得内层被放大的加密底噪 $\|E'_{total,c}\| = \| \sum_{i \in \mathcal{T}_{c,1}} (L_{c,i}^{(\mathcal{T}_{c,1})} e'_{c,i,1} + e'_{c,i,2}) \| < \Delta'/2$），**内层底噪 $E'_{total,c}$ 即在取整时被 100% 抹除！** CH_c 由此得到还原到外层环 $\mathcal{R}_{M,Q}$ 的**簇级近似解密份额** b'_c：

$$b'_c = \left\lfloor \frac{C'_{all,c,0} + (-a'_c u'_c) \cdot dk_c}{\Delta'} \right\rceil \pmod{Q} = c_1 \cdot S^{(c)} + \sum_{i \in \mathcal{T}_{c,1}} n_{c,i} \pmod{Q}$$

- CH_c 将这个已经完成内层解密取整、还原到模 Q 的多项式 $b'_c \in \mathcal{R}_{M,Q}$ 上传给云服务器 CS。

阶段 4：云服务器跨簇合成与外层 CKKS 浮点解码（`Cloud-CKKS-Decode`）

云服务器 CS 收到所有 C 个簇头上传的 b'_c 后，结合外层密文首项 c_0 完成最终闭环：

1. **跨簇加法合成外层近似明文多项式** $m'_{agg}(X)$：

1. $$
\begin{aligned}
m'_{agg}(X) &= c_0 + \sum_{c=1}^{C} b'_c \pmod{Q} \\
&= c_0 + c_1 \cdot \underbrace{\left(\sum_{c=1}^{C} S^{(c)} \right)}_{=S} + \sum_{c=1}^{C} \sum_{i \in \mathcal{T}_{c,1}} n_{c,i} \pmod{Q} \\
&= M_{agg}(X) + e_{CT} + \sum_{c=1}^{C} \sum_{i \in \mathcal{T}_{c,1}} n_{c,i} \pmod{Q}
\end{aligned}
$$

2. **执行 CKKS 解码输出全局浮点梯度：** 只要外层模数满足 $\| M_{agg}(X) + e_{CT} + \sum_{c,i} n_{c,i} \| < Q/2$，服务器调用 `CKKS.Decode`（正演变换 ϕ 并除以 Δ_{ckks}）：

1. $$
\mathbf{G}_{global} = \Delta_{ckks}^{-1} \cdot \phi\big(m'_{agg}(X)\big) = \sum_{c,i} \mathbf{g}_{c,i} + \underbrace{\frac{\phi\left(e_{CT} + \sum_{c=1}^{C} \sum_{i \in \mathcal{T}_{c,1}} n_{c,i}\right)}{\Delta_{ckks}}}_{\text{微小浮点近似误差 } \approx 10^{-8}}
$$

2. 由於拉格朗日系数被彻底隔绝在 $\sum n_{c,i}$ 之外，且 $\Delta_{ckks} \gg \sum n_{c,i}$，最终恢复出的全局梯度 $\mathbf{G}_{global}$ 保持了极高的浮点精度，同时完美保护了各终端梯度与簇级模型的隐私()。

3. 方案一实体交互流程汇总表

阶段	发送方 →\rightarrow→ 接收方	传输内容与符号	通信规模与所在环	物理目的
阶段 1	$P_{c,i} \rightarrow \text{CH}_c \rightarrow \text{CS}$	外层梯度密文 $CT_{c,i} = (c_{c,i,0}, c_{c,i,1})$	2 个 $\mathcal{R}_{M,Q}$ 多项式	逐级同态聚合全局梯度密文 $CT=(
阶段 1	$\text{CS} \rightarrow \text{CH}_c \rightarrow P_{c,i}$	外层聚合密文第二项 c_1	1 个 $\mathcal{R}_{M,Q}$ 多项式	触发分布式门限解密流程
阶段 2	$P_{c,i} \rightarrow P_{c,j}$ (同簇内)	内层双钥切片 $(ekShare^{(c)}_{j,i,1}, ekShare^{(c)}_{j,i,2})$	2 个 $\mathcal{R}_{M,Q'}$ 环元素	簇内局域网备份内层双私钥，复杂
阶段 2	$P_{c,i} \rightarrow \text{CH}_c$	内层双密文 $(CTb_{c,i}, CTn_{c,i})$	2 个 $\mathcal{R}_{M,Q'}$ 密文	密态提交外层解密份额 $b_{c,i}$ 与
阶段 2	$\text{CH}_c \rightarrow P_{c,j}$ (同簇内)	簇内在线名单 $\mathcal{T}_{c,1}$ 及拉格朗日系数 $\{L_{c,i}^{(\mathcal{T}_{c,1})}\}$	$O(n_c)$ 个标量/系数	同步本簇第一回合成功参与的终端
阶段 3	$P_{c,j} \rightarrow \text{CH}_c$	捆绑内层密钥份额 $dkShare_{c,j}$	1 个 $\mathcal{R}_{M,Q'}$ 环元素	帮助簇头重构 $dk_c = S^{(c)}_{in
阶段 3	$\text{CH}_c \rightarrow \text{CS}$	**脱壳后的簇级近似解密份额 b'_c**	**1 个 $\mathcal{R}_{M,Q}$ 多项式**	**消除跨广域网的 Q' 模数膨胀与**

方案二：面向 CKKS 的非共谋双服务器架构（Twin-Server Non-Colluding CKKS-ATASSES）

1. 威胁模型（Threat Model）

1. **系统拓扑与实体**：系统包含 N 个终端客户端（Clients $P_i, i \in [1, N]$）以及两个部署在独立安全域的云服务器——**聚合服务器（Aggregation Server, $\mathcal{S}_A$）与辅助密钥服务器（Helper / Key Server, $\mathcal{S}_H$）**()。

1. **单轮异步与掉线容忍（One-Shot & Dropout Resilience）**：客户端在解密阶段仅需执行单轮（One-Shot）无状态上传即可立刻离线休眠()；允许任意最多 $N - T$ 个客户端随时掉线（即在线集合 $\mathcal{T}$ 满足 $|\mathcal{T}| \geq T$）()；客户端之间零点对点（Zero-P2P）通信()。

1. **非共谋与抗合谋能力（Non-Collusion & Corruption）**：$\mathcal{S}_A$ 与 $\mathcal{S}_H$ 均为半诚实（Honest-but-Curious）服务器，且严格满足**双服务器非共谋假设（Non-Collusion Assumption）**()。允许任意服务器（$\mathcal{S}_A$ 或 $\mathcal{S}_H$ 之一）与最多 $T-1$ 个被腐化的客户端合谋，诚实客户端的个体梯度 $\mathbf{g}_i$ 与外层总私钥 S 均不会泄露()。

2. 方案流程与闭环公式推导

阶段 0：系统初始化与外层私钥切片（`Twin-CKKS-Setup`）

1. 外层全局总私钥为 $S \in \mathcal{R}_{M,Q}$，外层全局公钥为 $PK = (p_0, p_1) = (-a \cdot S + e_{pk},\ a) \in \mathcal{R}_{M,Q}^2$ ()。

1. 每个客户端 P_i 持有总私钥 S 在环 $\mathcal{R}_{M,Q}$ 上的 (T, N)-Shamir 份额 $s_i \in \mathcal{R}_{M,Q}$。对于任意大小 $|\mathcal{T}| \geq T$ 的在线客户端集合，拉格朗日系数 $L_i^{(\mathcal{T})}$ 满足 $\sum_{i \in \mathcal{T}} L_i^{(\mathcal{T})} \cdot s_i = S \pmod{Q}$ ()。

1. 公共随机源（CRS）每轮同步生成公共多项式基底 $a' \cdot u' \in \mathcal{R}_{M,Q}$ ()。

阶段 1：外层 CKKS 梯度加密与聚合（`Outer-CKKS-Agg`）

1. 每个客户端 P_i 将本地浮点梯度 $\mathbf{g}_i$ 编码为 $m_i(X) = \lfloor \Delta_{ckks} \cdot \phi^{-1}(\mathbf{g}_i) \rceil \in \mathcal{R}_{M,Q}$ ()，并用外层全局公钥 $PK = (p_0, p_1)$ 加密为 $CT_i = (c_{i,0}, c_{i,1})$ 上传给聚合服务器 $\mathcal{S}_A$ ()：

1. $$\begin{cases} c_{i,0} = p_0 \cdot u_i + m_i(X) + e_{i,0} \pmod{Q} \\ c_{i,1} = p_1 \cdot u_i + e_{i,1} \pmod{Q} \end{cases}$$

2. $\mathcal{S}_A$ 执行同态加法聚合得到全局梯度密文 $CT = (c_0, c_1) = (\sum_i c_{i,0}, \sum_i c_{i,1}) \pmod{Q}$，满足 $c_0 + c_1 \cdot S = M_{agg}(X) + e_{CT} \pmod{Q}$ ()。

1. $\mathcal{S}_A$ 将 c_1 广播给所有客户端，请求解密 $b = c_1 \cdot S \pmod{Q}$ ()。

阶段 2：客户端单轮上传两份掩码密文（`Client-OneShot-Upload`）

收到 c_1 后，在线客户端 $P_i \in \mathcal{T}$ 无需放大模数（直接在外层原环 $\mathcal{R}_{M,Q}$ 上操作），也无需与其他客户端通信，仅执行以下单轮操作：

1. **计算外层 CKKS 部分解密份额与混淆噪声：**

1. $b_i = c_1 \cdot s_i \pmod{Q}, \quad n_i \xleftarrow{\$} [-B_{sm}, B_{sm}]$
2. **利用双种子生成环 $\mathcal{R}_{M,Q}$ 上的两把掩码私钥：** 借鉴 *26-SFLBR.pdf* 与 *17-SecAgg.pdf* 的双伪随机种子思想()，P_i 随机采样两个 128-bit 短种子 $(\text{seed}_{i,1}, \text{seed}_{i,2})$，通过伪随机发生器 PRG 在外层环 $\mathcal{R}_{M,Q}$ 上直接展开生成两把均匀随机的同维掩码私钥：

1. $ek_{i,1} = \text{PRG}(\text{seed}_{i,1}) \in \mathcal{R}_{M,Q}, \quad ek_{i,2} = \text{PRG}(\text{seed}_{i,2}) \in \mathcal{R}_{M,Q}$
2. *(关键优势：因为 $ek_{i,1}, ek_{i,2}$ 在 $\mathcal{R}_{M,Q}$ 上均匀随机且每次迭代刷新，$(a'u') \cdot ek_{i,1}$ 直接构成环上的完美一次性密码本/PRF 掩码，因此**无需添加内层 RLWE 底噪 e'，也彻底省去了内层模数 Q' 与缩放因子 Δ'！**)*

1. **构造模 Q 同一环上的两份掩码密文：**
    - **通道 1（保护解密份额 b_i）：**
    - $c'_{b,i,0} = (a' \cdot ek_{i,1}) \cdot u' + b_i \pmod{Q}$
    - **通道 2（保护混淆噪声 n_i）：**
    - $c'_{n,i,0} = (a' \cdot ek_{i,2}) \cdot u' + n_i \pmod{Q}$
    - 公共第二项固定为 $c'_1 = -a' \cdot u' \pmod{Q}$。

1. **单轮分流发送（发完即走）：**
    - $P_i \to \mathcal{S}_A$（发往聚合服务器）：上传两份掩码密文 $(c'_{b,i,0},\ c'_{n,i,0}) \in \mathcal{R}^2_{M,Q}$。
    - $P_i \to \mathcal{S}_H$（通过 TLS 安全信道发往辅助服务器()）：上传两个短种子 $(\text{seed}_{i,1},\ \text{seed}_{i,2})$（仅 32 字节！）。

阶段 3：双服务器协同合成联合解密密钥 $dk = S_1 + S_2$（`Twin-Server-Coop`）

1. **聚合服务器 $\mathcal{S}_A$ 确定在线集合 $\mathcal{T}$ 并合成总掩码密文：** $\mathcal{S}_A$ 收齐在线客户端集合 $\mathcal{T}$（验证 $|\mathcal{T}| \geq T$）上传的 $(c'_{b,i,0}, c'_{n,i,0})$ 后，计算拉格朗日系数 $\{L_i^{(\mathcal{T})}\}_{i\in\mathcal{T}}$，并在模 Q 下执行**非对称线性同态组合**（通道 1 乘 $L_i^{(\mathcal{T})}$，通道 2 直接相加）()：

1. $$\begin{aligned} C'_{all,0} &= \sum_{i\in\mathcal{T}} \left(L_i^{(\mathcal{T})} \cdot c'_{b,i,0} + c'_{n,i,0} \right) \pmod{Q} \\ &= (a' \cdot u') \cdot \underbrace{\left(\sum_{i\in\mathcal{T}} L_i^{(\mathcal{T})} \cdot ek_{i,1} \right)}_{\text{第一总私钥 } S_1} + (a' \cdot u') \cdot \underbrace{\left(\sum_{i\in\mathcal{T}} ek_{i,2} \right)}_{\text{第二总私钥 } S_2} + \underbrace{\sum_{i\in\mathcal{T}} L_i^{(\mathcal{T})} \cdot b_i}_{=c_1 \cdot S = b} + \sum_{i\in\mathcal{T}} n_i \pmod{Q} \\ &= (a' \cdot u') \cdot (S_1 + S_2) + c_1 \cdot S + \sum_{i\in\mathcal{T}} n_i \pmod{Q} \end{aligned}$$

2. $\mathcal{S}_A$ 将本轮在线节点名单 $\mathcal{T}$ 发送给辅助服务器 $\mathcal{S}_H$ ()。

1. **辅助服务器 $\mathcal{S}_H$ 展开种子并合并两把总私钥 $dk = S_1 + S_2$：** $\mathcal{S}_H$ 收到名单 $\mathcal{T}$ 后，首先检查 $|\mathcal{T}| \geq T$ 是否成立（防止恶意 $\mathcal{S}_A$ 伪造单节点名单套取个体密钥()）。验证通过后，$\mathcal{S}_H$ 计算对应的拉格朗日系数 $\{L_i^{(\mathcal{T})}\}_{i\in\mathcal{T}}$，利用收到的种子展开 $ek_{i,1} = \mathrm{PRG}(\mathrm{seed}_{i,1})$ 和 $ek_{i,2} = \mathrm{PRG}(\mathrm{seed}_{i,2})$，分别算出第一把子密钥 S_1 和第二把子密钥 S_2，并将二者在模 Q 下**相加合并**为单一联合解密密钥 dk ()：

1. $$\begin{cases} S_1 = \sum_{i\in\mathcal{T}} L_i^{(\mathcal{T})} \cdot ek_{i,1} \pmod{Q} \\ S_2 = \sum_{i\in\mathcal{T}} ek_{i,2} \pmod{Q} \end{cases} \implies dk = S_1 + S_2 \pmod{Q}$$

2. $\mathcal{S}_H$ 仅将捆绑后的 $dk \in \mathcal{R}_{M,Q}$ 发送给 $\mathcal{S}_A$（由于 S_2 未知，$\mathcal{S}_A$ 无法从 dk 中分离出 S_1 或任何 $ek_{i,1}$）()。

阶段 4：聚合服务器 $\mathcal{S}_A$ 一步消元与 CKKS 浮点解码（`Final-CKKS-Decode`）

1. **一步消去掩码项并得到解密结果：**

2. $\mathcal{S}_A$ 利用收到的 $dk = S_1 + S_2$ 和公共第二项 $C'_{all,1} = -a' \cdot u'$，直接在模 Q 下完成线性消元：

1. $$\begin{aligned} b' &= C'_{all,0} + C'_{all,1} \cdot dk \pmod{Q} \\ &= \left[(a' \cdot u') \cdot (S_1 + S_2) + c_1 \cdot S + \sum_{i\in\mathcal{T}} n_i \right] + (-a' \cdot u') \cdot (S_1 + S_2) \pmod{Q} \\ &= c_1 \cdot S + \sum_{i\in\mathcal{T}} n_i \pmod{Q} \end{aligned}$$

2. **合并外层首项 c_0 并解码还原全局浮点梯度：**

3. $\mathcal{S}_A$ 将 b' 与外层密文首项 c_0 相加：

1. $$m'_{agg}(X) = c_0 + b' = M_{agg}(X) + e_{CT} + \sum_{i\in\mathcal{T}} n_i \pmod{Q}$$

2. 最后调用 `CKKS.Decode` 除以 Δ_{ckks} 完成解码，无损输出高精度全局浮点梯度()：

1. $$\mathbf{G}_{global} = \Delta_{ckks}^{-1} \cdot \phi\left(m'_{agg}(X)\right) = \sum_{i=1}^{N} \mathbf{g}_i + \frac{\phi\left(e_{CT} + \sum_{i\in\mathcal{T}} n_i\right)}{\Delta_{ckks}}$$

3. 方案二实体交互流程汇总表

阶段	发送方 →\rightarrow→ 接收方	传输内容与符号	通信规模与所在环	物理目的
阶段 1	$P_i \rightarrow \mathcal{S}_A$	外层梯度密文 $CT_i = (c_{i,0}, c_{i,1})$	2 个 $\mathcal{R}_{M,Q}$ 多项式	同态累加全局梯
阶段 1	$\mathcal{S}_A \rightarrow P_i$	外层聚合密文第二项 c_1	1 个 $\mathcal{R}_{M,Q}$ 多项式	触发客户端计算
阶段 2	$P_i \rightarrow \mathcal{S}_A$	同维双路掩码密文 $(c'_{b,i,0}, c'_{n,i,0})$	2 个 $\mathcal{R}_{M,Q}$ 多项式（**无 Q' 膨胀**）	分别封装 b_i
阶段 2	$P_i \rightarrow \mathcal{S}_H$ (安全信道)	双通道伪随机种子 $(\text{seed}_{i,1}, \text{seed}_{i,2})$	**$O(1)$ 常数级（32 Bytes）**	委托 $\mathcal{ **$O(N^2)$ P2P
阶段 3	$\mathcal{S}_A \rightarrow \mathcal{S}_H$	本轮在线客户端集合名单 $\mathcal{T}$	$O(N)$ 个 ID 整数	告知 $\mathcal{ 算 $L_i^{(\math
阶段 3	$\mathcal{S}_H \rightarrow \mathcal{S}_A$	焊接后的联合总私钥 $dk = S_1 + S_2$	1 个 $\mathcal{R}_{M,Q}$ 多项式	协助 $\mathcal{ c_1 S + \sum n_

- **结合你已上传的 16 篇文献构建创新点**：是否需要将上述 **方案一（层次架构）** 或 **方案二（双服务器架构）** 与你上传的 *25-FedPHE*（包级 Top-k 稀疏化()）、*26-PBFL / 26-SFLBR*（密态余弦相似度/中位数拜占庭鲁棒过滤()）或 *26-VLSFL*（同态哈希防篡改验证）深度缝合，为您打造一套完整的 SCI 一区/CCF-A 级论文核心章节（含算法伪代码与安全证明）？

- **实验仿真代码落地**：需要为您编写一套基于 Python（使用 `TenSEAL` 或 `Pyfhel` 模拟外层 CKKS + 内层联合解密密钥 S_1, S_2 消元）的可运行验证脚本，以实测其解密精度误差（如 10^{-9} 量级）吗？